

SR-Aging_brain-plots

Cooper Joseph Sharp

2025-12-19

Purpose and note on circularity

These plots are **descriptive** summaries of values extracted from clusters identified in whole-brain analyses. Because the cluster definition and the extraction come from the same dataset/effect, the relationships shown here are **selection-biased (circular)** and should not be interpreted as independent effect size estimates.

1. Load libraries

```
packages <- c(
  "dplyr",
  "ggplot2",
  "readr",
  "tibble"
)

# Install missing packages (comment out if you prefer not to install during knitting)
installed <- rownames(installed.packages())
for (pkg in packages) {
  if (!pkg %in% installed) {
    install.packages(pkg, repos = "https://cloud.r-project.org")
  }
}

# Load packages
invisible(lapply(packages, library, character.only = TRUE))
```

2. Data import and preparation

```
# Read the combined CSV file
# (If you knit from a different directory, consider using an absolute path or `here::here()`.)
df <- readr::read_csv("extractions-retest-copy.csv", show_col_types = FALSE)

# Quick sanity checks
stopifnot(is.data.frame(df))
```

2.1 Helper utilities (weights + plotting)

```
# Inverse-variance weights with a floor to prevent infinite / extreme weights
make_invvar_weights <- function(var_vec, var_floor = 1e-6) {
  1 / pmax(var_vec, var_floor)
}
```

```

# Print a compact weight diagnostic (useful when plots "don't look right")
summarize_weights <- function(w) {
  tibble::tibble(
    n = length(w),
    n_nonfinite = sum(!is.finite(w)),
    min = min(w, na.rm = TRUE),
    p01 = as.numeric(stats::quantile(w, 0.01, na.rm = TRUE)),
    p05 = as.numeric(stats::quantile(w, 0.05, na.rm = TRUE)),
    median = stats::median(w, na.rm = TRUE),
    p95 = as.numeric(stats::quantile(w, 0.95, na.rm = TRUE)),
    p99 = as.numeric(stats::quantile(w, 0.99, na.rm = TRUE)),
    max = max(w, na.rm = TRUE)
  )
}

# Partial residual (added-variable style) for age_demeaned from a fitted lm()
partial_residual_age <- function(model, df, x = "age_demeaned") {
  b <- coef(model)[[x]]
  stats::residuals(model) + b * df[[x]]
}

# Raw scatter: unweighted fit
plot_scatter_raw_unweighted <- function(df, outcome_col, outcome_label) {
  ggplot(df, aes(x = age_demeaned, y = .data[[outcome_col]])) +
    geom_point(alpha = 0.55) +
    geom_smooth(method = "lm", se = TRUE) +
    theme_minimal() +
    labs(
      x = "age_demeaned",
      y = outcome_label,
      title = paste0("Raw scatter (unweighted): ", outcome_label),
      caption = "Selection-biased (circular) visualization; do not treat as an independent effect size."
    )
}

# Raw scatter: weighted fit + point size reflects weights
plot_scatter_raw_weighted <- function(df, outcome_col, weight_col, outcome_label) {
  ggplot(df, aes(x = age_demeaned, y = .data[[outcome_col]])) +
    geom_point(aes(size = sqrt(.data[[weight_col]])), alpha = 0.55) +
    geom_smooth(aes(weight = .data[[weight_col]]), method = "lm", se = TRUE) +
    scale_size_continuous(guide = "none") +
    theme_minimal() +
    labs(
      x = "age_demeaned",
      y = outcome_label,
      title = paste0("Raw scatter (weighted): ", outcome_label),
      caption = "Point size reflects inverse-variance weight; smooth uses the same weights. Selection-biased."
    )
}

# Added-variable (partial residual) scatter: unweighted
plot_added_variable_unweighted <- function(model_unw, df, outcome_label) {
  pr <- partial_residual_age(model_unw, df)

```

```

ggplot(tibble::tibble(age_demeaned = df$age_demeaned, pr = pr),
      aes(x = age_demeaned, y = pr)) +
  geom_point(alpha = 0.55) +
  geom_smooth(method = "lm", se = TRUE) +
  theme_minimal() +
  labs(
    x = "age_demeaned",
    y = paste0("Partial residuals: ", outcome_label),
    title = paste0("Added-variable plot (unweighted): ", outcome_label),
    caption = "Partial residuals are computed from the unweighted model. Selection-biased (circular)."
  )
}

# Added-variable (partial residual) scatter: weighted
plot_added_variable_weighted <- function(model_w, df, weight_col, outcome_label) {
  pr <- partial_residual_age(model_w, df)

  ggplot(tibble::tibble(age_demeaned = df$age_demeaned, pr = pr, w = df[[weight_col]]),
        aes(x = age_demeaned, y = pr)) +
    geom_point(aes(size = sqrt(w)), alpha = 0.55) +
    geom_smooth(aes(weight = w), method = "lm", se = TRUE) +
    scale_size_continuous(guide = "none") +
    theme_minimal() +
    labs(
      x = "age_demeaned",
      y = paste0("Partial residuals: ", outcome_label),
      title = paste0("Added-variable plot (weighted): ", outcome_label),
      caption = "Partial residuals and smooth correspond to the weighted model. Selection-biased (circular)."
    )
}

# Correlation heatmap for a set of variables (coerces factors to numeric codes)
plot_corr_heatmap <- function(df,
                              cols,
                              title = "Correlation heatmap",
                              alpha = 0.05,
                              adjust_method = "BH",
                              threshold = TRUE,
                              show_numbers = c("significant", "none", "all")) {

  show_numbers <- match.arg(show_numbers)

  dat <- df[, cols, drop = FALSE]

  # Coerce non-numeric to numeric (factors -> integer codes; characters -> factor -> codes)
  for (nm in names(dat)) {
    if (!is.numeric(dat[[nm]])) {
      dat[[nm]] <- as.numeric(as.factor(dat[[nm]]))
    }
  }

  k <- ncol(dat)

```

```

rn <- colnames(dat)

r <- matrix(NA_real_, k, k, dimnames = list(rn, rn))
p <- matrix(NA_real_, k, k, dimnames = list(rn, rn))
n <- matrix(NA_integer_, k, k, dimnames = list(rn, rn))

for (i in seq_len(k)) {
  for (j in i:k) {
    ok <- stats::complete.cases(dat[[i]], dat[[j]])
    nij <- sum(ok)
    n[i, j] <- nij
    n[j, i] <- nij

    if (nij >= 3) {
      rij <- stats::cor(dat[[i]][ok], dat[[j]][ok])
      r[i, j] <- rij
      r[j, i] <- rij

      if (i == j) {
        p[i, j] <- NA_real_
        p[j, i] <- NA_real_
      } else {
        tval <- rij * sqrt((nij - 2) / (1 - rij^2))
        pij <- 2 * stats::pt(-abs(tval), df = nij - 2)
        p[i, j] <- pij
        p[j, i] <- pij
      }
    }
  }
}

# Multiple-comparisons correction across unique off-diagonal pairs
p_adj <- p
upper_idx <- which(upper.tri(p), arr.ind = TRUE)
p_vec <- p[upper_idx]
keep <- is.finite(p_vec)
p_vec_adj <- rep(NA_real_, length(p_vec))
p_vec_adj[keep] <- stats::p.adjust(p_vec[keep], method = adjust_method)

# Fill symmetric adjusted p-values
p_adj[upper_idx] <- p_vec_adj
p_adj[t(upper_idx)] <- p_vec_adj

# Long format for plotting
long <- expand.grid(Var1 = rn, Var2 = rn, stringsAsFactors = FALSE)
long$r <- as.vector(r[cbind(match(long$Var1, rn), match(long$Var2, rn))])
long$p_adj <- as.vector(p_adj[cbind(match(long$Var1, rn), match(long$Var2, rn))])
long$sig <- is.finite(long$p_adj) & (long$p_adj < alpha)

# Keep diagonal always visible
diag_mask <- long$Var1 == long$Var2
long$r_plot <- ifelse(diag_mask, 1, long$r)

```

```

if (threshold) {
  long$r_plot <- ifelse(diag_mask | long$sig, long$r_plot, NA_real_)
}

if (show_numbers == "none") {
  long$lab <- ""
} else if (show_numbers == "all") {
  long$lab <- ifelse(is.finite(long$r), sprintf("%.2f", long$r), "")
  long$lab <- ifelse(diag_mask, "1.00", long$lab)
} else { # significant
  long$lab <- ifelse(diag_mask | long$sig, sprintf("%.2f", long$r_plot), "")
}

ggplot(long, aes(x = Var1, y = Var2, fill = r_plot)) +
  geom_tile(color = "white", linewidth = 0.3) +
  geom_text(aes(label = lab), size = 3) +
  coord_fixed() +
  scale_fill_gradient2(
    limits = c(-1, 1),
    midpoint = 0,
    na.value = "grey95"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(
    title = title,
    x = NULL,
    y = NULL,
    fill = "r",
    caption = if (threshold) {
      paste0("Cells shown only if FDR-adjusted p < ", alpha, " (method: ", adjust_method, ").")
    } else {
      "All pairwise correlations shown (no multiple-comparisons thresholding)."
    }
  )
}

# ---- Sextile (6-bin) summaries + bar plots ----

# Weighted mean / SE for descriptive summaries.
# Uses an effective sample size (n_eff) and an unbiased-ish weighted variance.
wtd_mean_se <- function(y, w) {
  ok <- is.finite(y) & is.finite(w) & (w > 0)
  y <- y[ok]; w <- w[ok]

  if (length(y) == 0) return(list(mean = NA_real_, se = NA_real_, n = 0))

  mu <- sum(w * y) / sum(w)
  sw <- sum(w)
  sw2 <- sum(w^2)
  n_eff <- if (sw2 > 0) (sw^2 / sw2) else length(y)

  # Denominator for an unbiased-ish weighted variance

```

```

den <- sw - (sw2 / sw)
v <- if (den > 0) sum(w * (y - mu)^2) / den else NA_real_
se <- if (is.finite(v) && n_eff > 1) sqrt(v / n_eff) else NA_real_

list(mean = mu, se = se, n = length(y))
}

summarize_sexiles <- function(df, y, series_label, w = NULL, x = "age_demeaned", n_bins = 6) {
  tmp <- tibble::tibble(x = df[[x]], y = y)
  tmp <- tmp %>%
    dplyr::filter(is.finite(x), is.finite(y)) %>%
    dplyr::mutate(bin = dplyr::ntile(x, n_bins))

  if (is.null(w)) {
    tmp %>%
      dplyr::group_by(bin) %>%
      dplyr::summarise(
        n = dplyr::n(),
        age_min = min(x),
        age_max = max(x),
        x_mean = mean(x),
        y_mean = mean(y),
        y_se = stats::sd(y) / sqrt(n), # SEM
        .groups = "drop"
      ) %>%
      dplyr::mutate(
        series = series_label,
        bin_label = paste0(round(age_min, 2), " to ", round(age_max, 2))
      )
  } else {
    # Align weights to the same finite rows used above
    ok <- is.finite(df[[x]]) & is.finite(y) & is.finite(w) & (w > 0)
    tmp <- tibble::tibble(x = df[[x]][ok], y = y[ok], w = w[ok]) %>%
      dplyr::mutate(bin = dplyr::ntile(x, n_bins))

    tmp %>%
      dplyr::group_by(bin) %>%
      dplyr::summarise(
        n = dplyr::n(),
        age_min = min(x),
        age_max = max(x),
        x_mean = mean(x),
        y_mean = sum(w * y) / sum(w),
        y_se = sqrt(1 / sum(w)), # SEM for inverse-variance weighted mean
        .groups = "drop"
      ) %>%
      dplyr::mutate(
        series = series_label,
        bin_label = paste0(round(age_min, 2), " to ", round(age_max, 2))
      )
  }
}

```

```

plot_sexstile_bars <- function(summary_df, outcome_label, title_prefix) {
  # Expect summary_df to contain: bin (1-6), bin_label, y_mean, y_se
  summary_df$bin <- factor(summary_df$bin, levels = 1:6)

  # Use bin_label for x-axis labels (age ranges within each sextile)
  bin_labels <- summary_df %>%
    dplyr::distinct(bin, bin_label) %>%
    dplyr::arrange(bin)

  ggplot(summary_df, aes(x = bin, y = y_mean)) +
    geom_col(width = 0.8) +
    geom_errorbar(aes(ymin = y_mean - y_se, ymax = y_mean + y_se), width = 0.2) +
    scale_x_discrete(labels = bin_labels$bin_label) +
    theme_minimal() +
    theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
    labs(
      x = "Age sextile (ntile(age_demeaned, 6))",
      y = outcome_label,
      title = paste0(title_prefix, ": ", outcome_label),
      caption = "Bars show mean within sextile; error bars are  $\pm 1$  SEM. Selection-biased (circular)."
    )
}

```

3. DLPFC NPPI (rew > pun), Friend–Stranger

Outcome: mask.dlpfc_type.nppi_cnum.14_cname.rew.pun_F.S

Variance: var_mask.dlpfc_type.nppi_cnum.14_cname.rew.pun_F.S

Main predictor: age_demeaned

Covariates: sex + avg_tsnr_demeaned + avg_fdmean_demeaned + flip_demeaned + study_demeaned

```

# ---- DLPFC model (robust to name-repair on import) ----
# We expect readr::read_csv() may "repair" duplicate column names by appending ...1, ...2, etc.
# This helper finds the single best-matching column name in df.

find_one_col <- function(pattern, df_names = names(df)) {
  hits <- grep(pattern, df_names, value = TRUE)
  if (length(hits) == 1) return(hits)
  if (length(hits) == 0) stop(paste0("No columns match pattern: ", pattern))
  stop(paste0("Multiple columns match pattern: ", pattern, "
", paste(hits, collapse = "
"))))
}

# Match either exact name OR name with readr's ...<n> suffix.
# Allow either "." or "-" as the separator because some exports use hyphens.
outcome_dlpfc <- find_one_col("^mask[.-]dlpfc_type[.-]nppi_cnum[.-]14_cname[.-]rew[.-]pun_F[.-]S([.]{3})")
var_dlpfc <- find_one_col("^var_mask[.-]dlpfc_type[.-]nppi_cnum[.-]14_cname[.-]rew[.-]pun_F[.-]S([.]{3})")
w_dlpfc <- "weights_dlpfc_type.nppi_cnum.14_cname.rew.pun_F.S"

# Confirm required columns exist (now using detected names)
needed <- c(outcome_dlpfc, var_dlpfc,
  "age_demeaned", "sex", "avg_tsnr_demeaned", "avg_fdmean_demeaned",
  "flip_demeaned", "study_demeaned")

```

```

missing <- setdiff(needed, names(df))
if (length(missing) > 0) stop(paste("Missing required columns:", paste(missing, collapse = ", ")))

# If sex is coded numerically but should be categorical, convert to factor.
# (Comment out if you are sure you want a linear numeric effect.)
if (!is.factor(df$sex)) {
  u <- sort(unique(df$sex))
  if (length(u) <= 4) df$sex <- factor(df$sex)
}

# Create inverse-variance weights with a small floor (prevents Inf weights if variance is ~0)
df[[w_dlpfc]] <- make_invvar_weights(df[[var_dlpfc]], var_floor = 1e-6)

# Weight diagnostics (helpful if a few observations dominate)
print(summarize_weights(df[[w_dlpfc]]))

## # A tibble: 1 x 9
##       n n_nonfinite      min      p01      p05  median      p95      p99      max
##   <int>      <int>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
## 1    245          0 0.0000104 0.0000149 0.0000397 0.000582 0.00791 0.0229 0.0297

# Fit model EXACTLY as specified, but with the detected outcome column name
fml_dlpfc <- as.formula(sprintf(
  "%s` ~ age_demeaned + sex + avg_tsnr_demeaned + avg_fdmean_demeaned + flip_demeaned + study_demeaned`
  outcome_dlpfc
))

# Weighted and unweighted versions (for side-by-side plots)
model2_w <- lm(fml_dlpfc, data = df, weights = df[[w_dlpfc]])
model2_unw <- lm(fml_dlpfc, data = df)

summary(model2_w)

##
## Call:
## lm(formula = fml_dlpfc, data = df, weights = df[[w_dlpfc]])
##
## Weighted Residuals:
##      Min       1Q   Median       3Q      Max
## -1.86117 -0.39443  0.00079  0.41926  1.72059
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -1.55381    1.73312  -0.897  0.370875
## age_demeaned  -0.36669    0.06776  -5.412 1.53e-07 ***
## sexM           8.18065    2.22728   3.673 0.000296 ***
## sex0           8.92245    8.05110   1.108 0.268887
## avg_tsnr_demeaned  0.41411    0.13237   3.128 0.001977 **
## avg_fdmean_demeaned 89.53394   21.32840   4.198 3.81e-05 ***
## flip_demeaned  -24.89236    7.33566  -3.393 0.000809 ***
## study_demeaned   50.87925   11.37542   4.473 1.20e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##

```

```
## Residual standard error: 0.6452 on 237 degrees of freedom
## Multiple R-squared: 0.17, Adjusted R-squared: 0.1455
## F-statistic: 6.936 on 7 and 237 DF, p-value: 1.616e-07
```

```
summary(model2_unw)
```

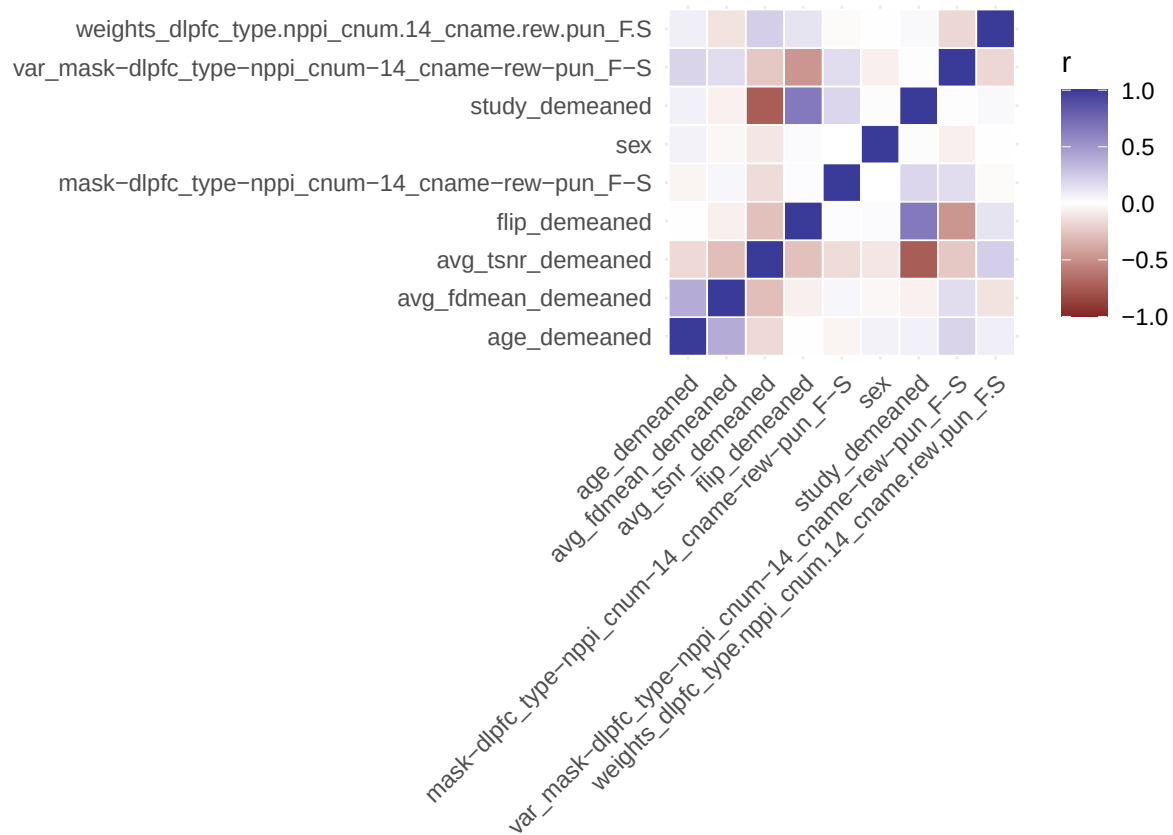
```
##
## Call:
## lm(formula = fml_dlpfc, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -207.39  -19.22   -1.03   17.88  256.64
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      0.6461      3.9292   0.164 0.869532
## age_demeaned     -0.3234      0.1870  -1.729 0.085134 .
## sexM              2.9920      6.4945   0.461 0.645445
## sex0              6.2827     24.0745   0.261 0.794343
## avg_tsnr_demeaned  0.5998      0.3983   1.506 0.133406
## avg_fdmean_demeaned 93.9966     47.1360   1.994 0.047281 *
## flip_demeaned     -29.0108     10.4441  -2.778 0.005912 **
## study_demeaned      75.3982     21.8946   3.444 0.000678 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 47.08 on 237 degrees of freedom
## Multiple R-squared: 0.078, Adjusted R-squared: 0.05077
## F-statistic: 2.864 on 7 and 237 DF, p-value: 0.006895
```

3.1 Plots for DLPFC

```
label_dlpfc <- "DLPFC NPPI (rew > pun), F-S"

# 0) Correlation heatmap among outcome + predictors (+ variance + weight)
# 0a) Correlation heatmap (all correlations; no cell labels)
plot_corr_heatmap(
  df,
  cols = c(outcome_dlpfc, "age_demeaned", "sex", "avg_tsnr_demeaned", "avg_fdmean_demeaned",
            "flip_demeaned", "study_demeaned", var_dlpfc, w_dlpfc),
  title = paste0("Correlation heatmap: ", label_dlpfc),
  threshold = FALSE,
  show_numbers = "none"
)
```

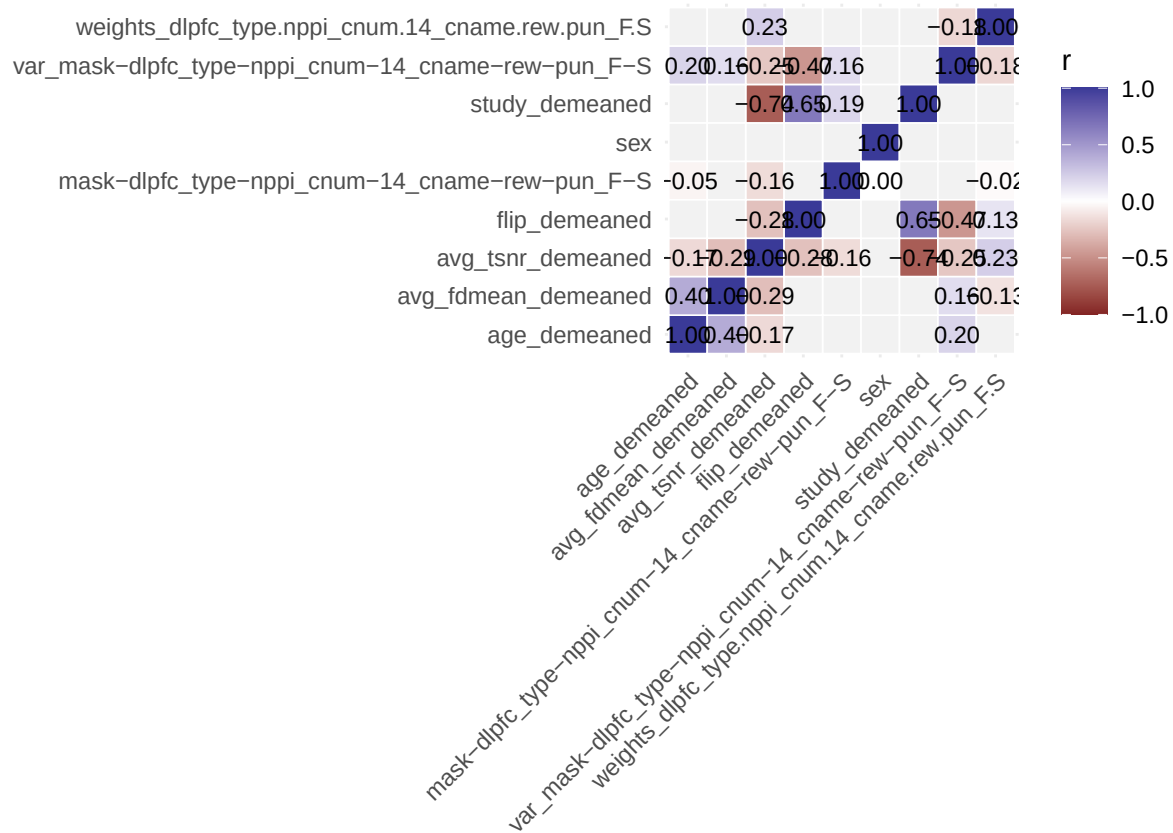
Correlation heatmap: DLPFC NPPI (rev



All pairwise correlations shown (no multiple-comparisons thresholding).

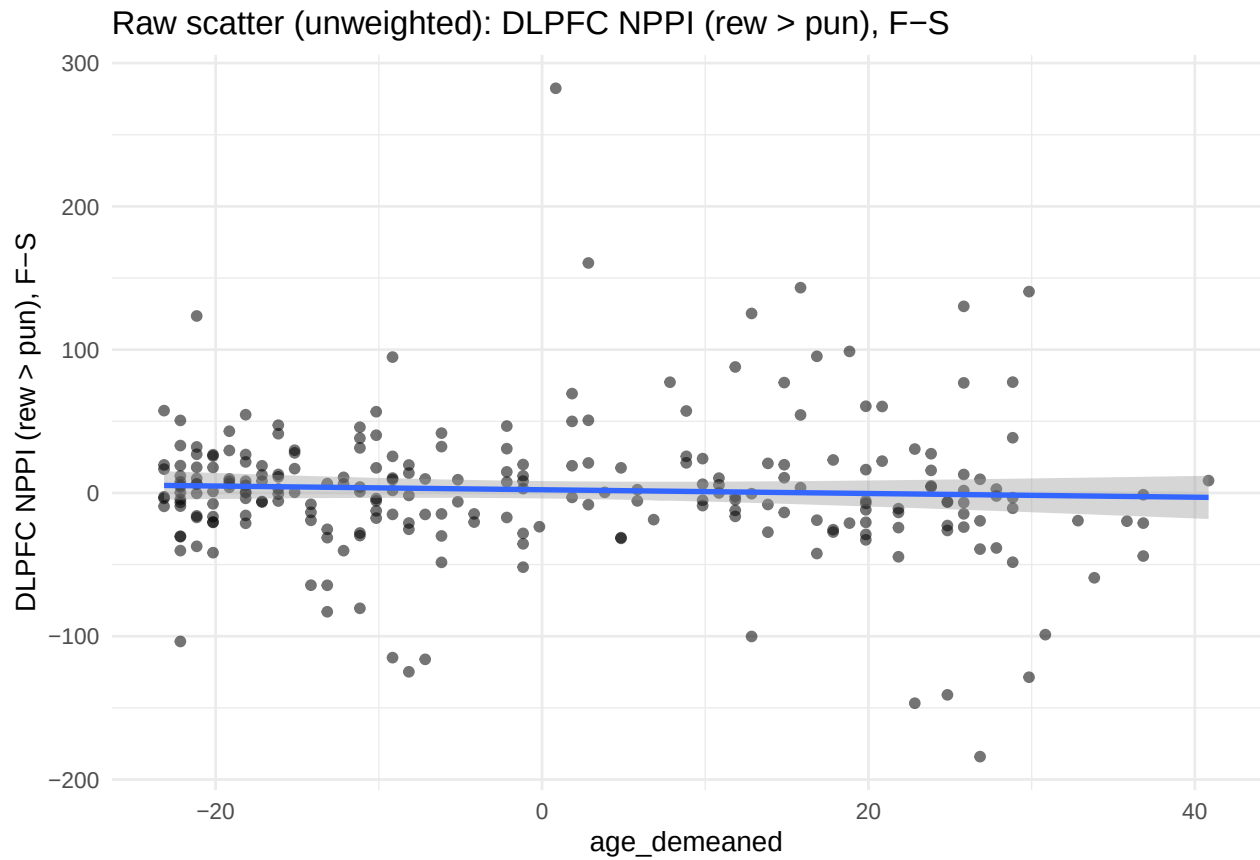
```
# Ob) Correlation heatmap (FDR-significant only; labels shown for significant cells)
plot_corr_heatmap(
  df,
  cols = c(outcome_dlpfc, "age_demeaned", "sex", "avg_tsnr_demeaned", "avg_fdmean_demeaned",
           "flip_demeaned", "study_demeaned", var_dlpfc, w_dlpfc),
  title = paste0(label_dlpfc, " (FDR-significant only)",
  threshold = TRUE,
  show_numbers = "significant"
)
```

DLPFC NPPI (rew > pun), F-S (FDR-s



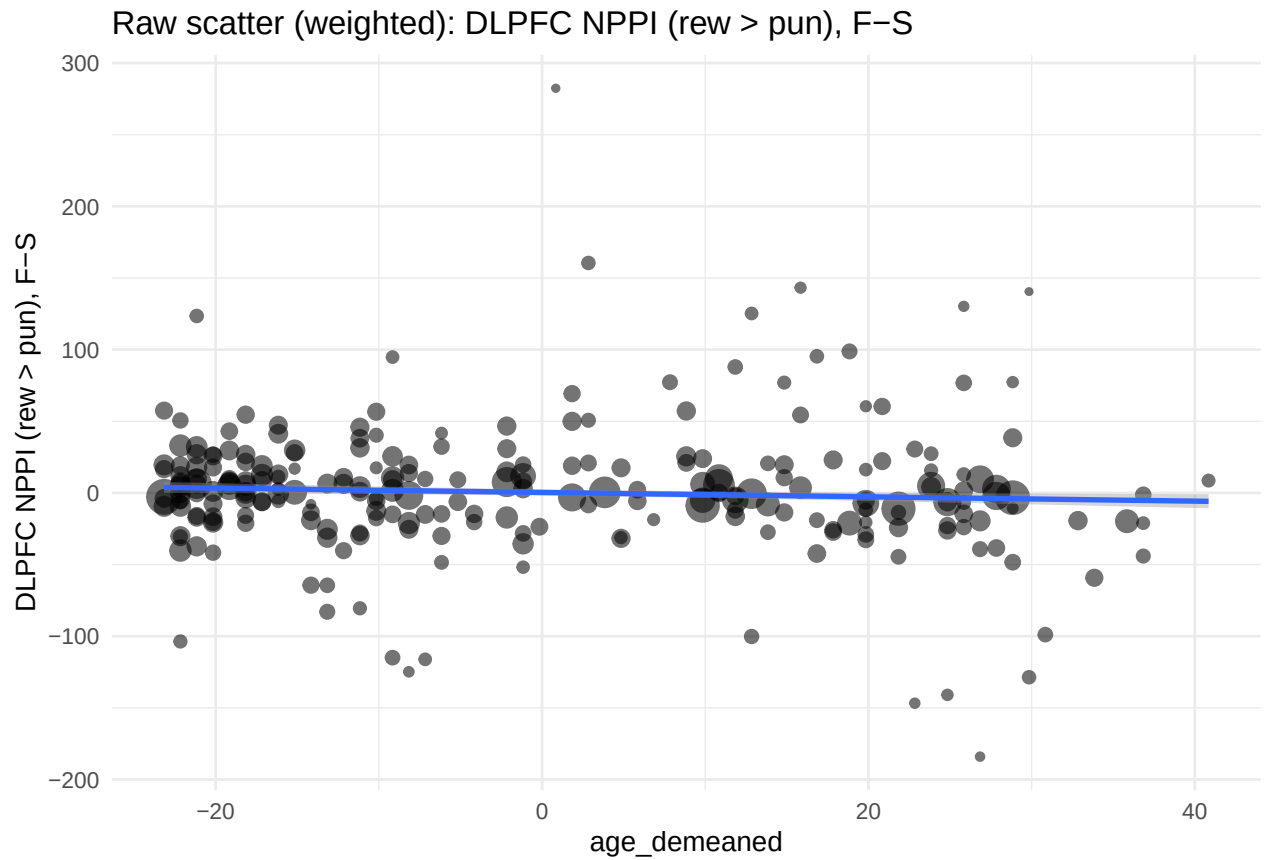
Cells shown only if FDR-adjusted $p < 0.05$ (method: BH).

```
# 1) Raw scatter: unweighted vs weighted
plot_scatter_raw_unweighted(df, outcome_dlpfc, label_dlpfc)
```



Selection-biased (circular) visualization; do not treat as an independent effect size.

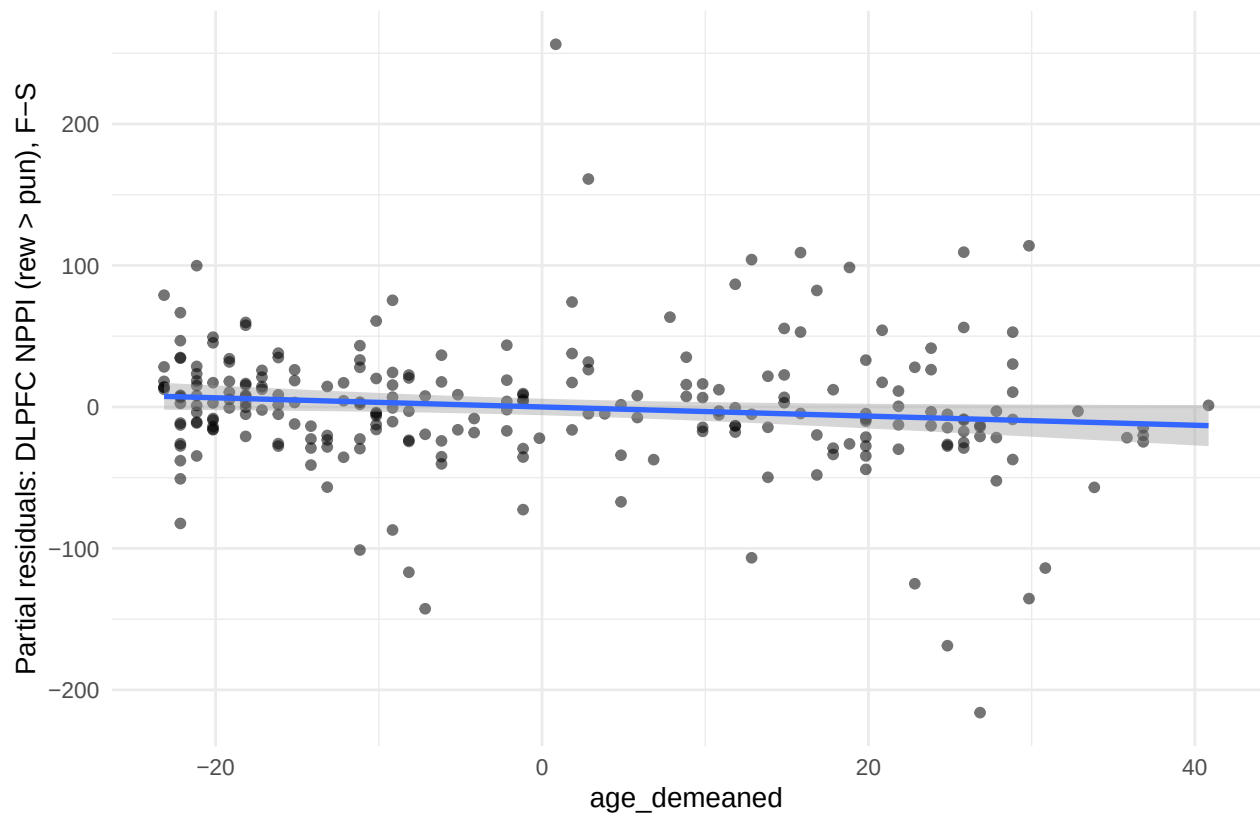
```
plot_scatter_raw_weighted(df, outcome_dlpfc, w_dlpfc, label_dlpfc)
```



Point size reflects inverse-variance weight; smooth uses the same weights. Selection-biased (circular).

```
# 2) Added-variable (partial residual) plots: unweighted vs weighted  
plot_added_variable_unweighted(model2_unw, df, label_dlpfc)
```

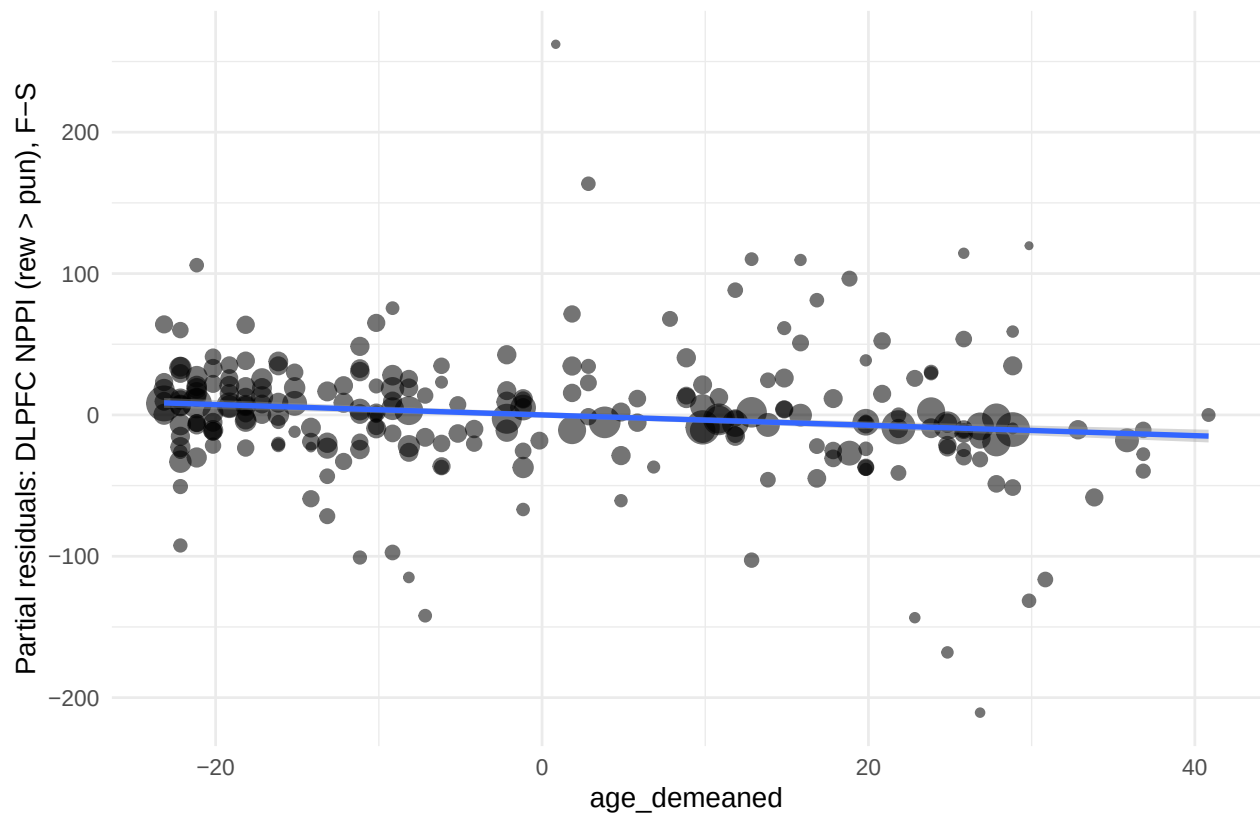
Added-variable plot (unweighted): DLPFC NPPI (rew > pun), F-S



Partial residuals are computed from the unweighted model. Selection-biased (circular).

```
plot_added_variable_weighted(model2_w, df, w_dlpfc, label_dlpfc)
```

Added-variable plot (weighted): DLPFC NPPI (rew > pun), F-S



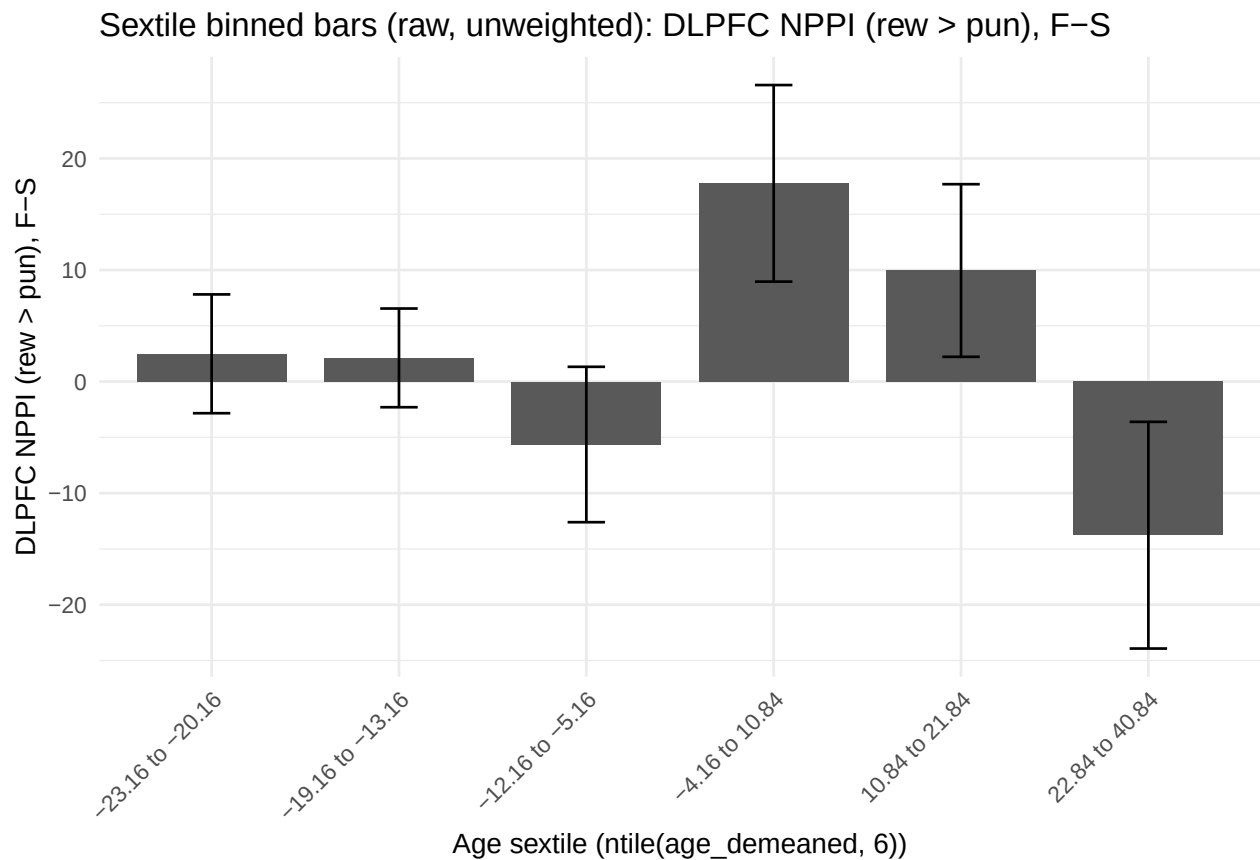
Partial residuals and smooth correspond to the weighted model. Selection-biased (circular).

3) Bar plots with age binned into sextiles (6 equal bins)

3a) Raw outcome bars (unweighted and weighted shown separately)

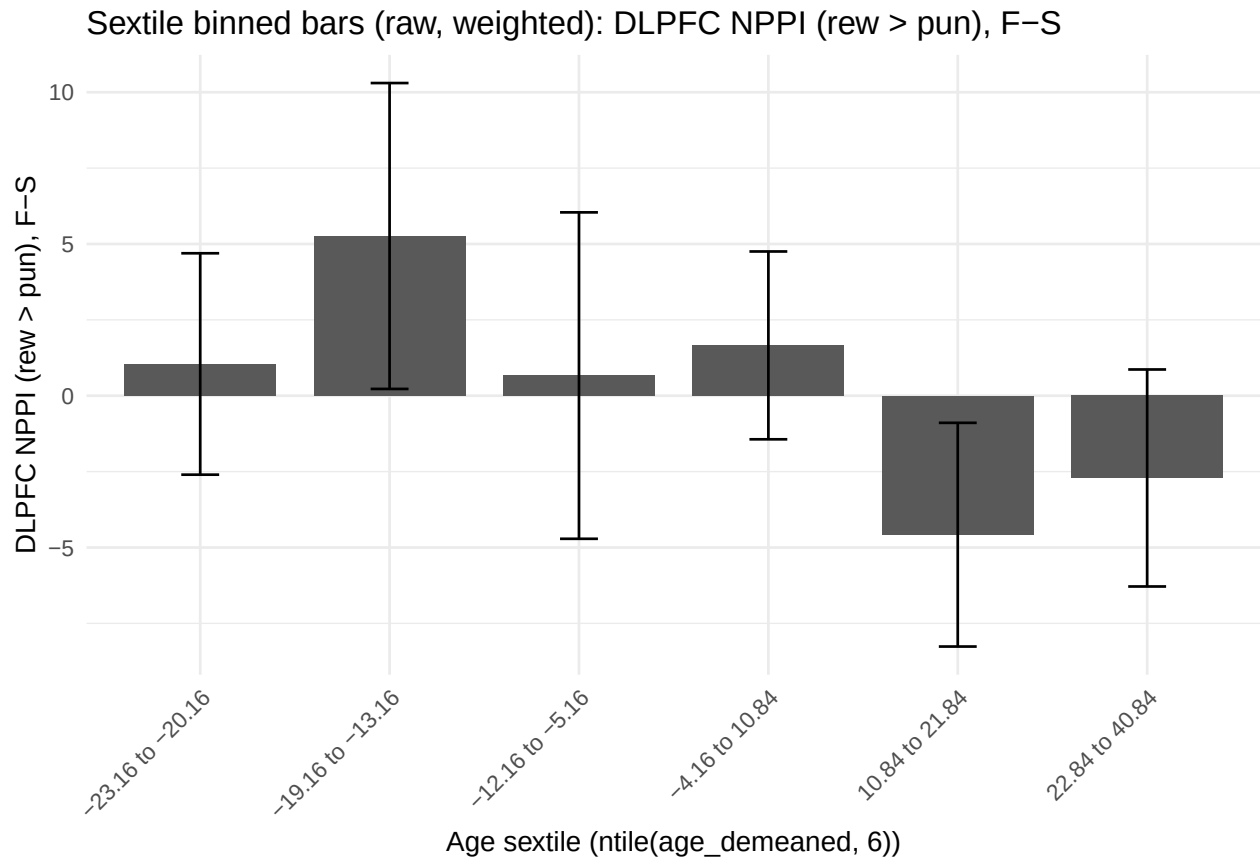
```
sum_raw_unw <- summarize_sextiles(df, y = df[[outcome_dlpfc]], series_label = "Unweighted mean")
```

```
plot_sextile_bars(sum_raw_unw, outcome_label = label_dlpfc, title_prefix = "Sextile binned bars (raw, u
```



Bars show mean within sextile; error bars are ± 1 SEM. Selection-biased (circular).

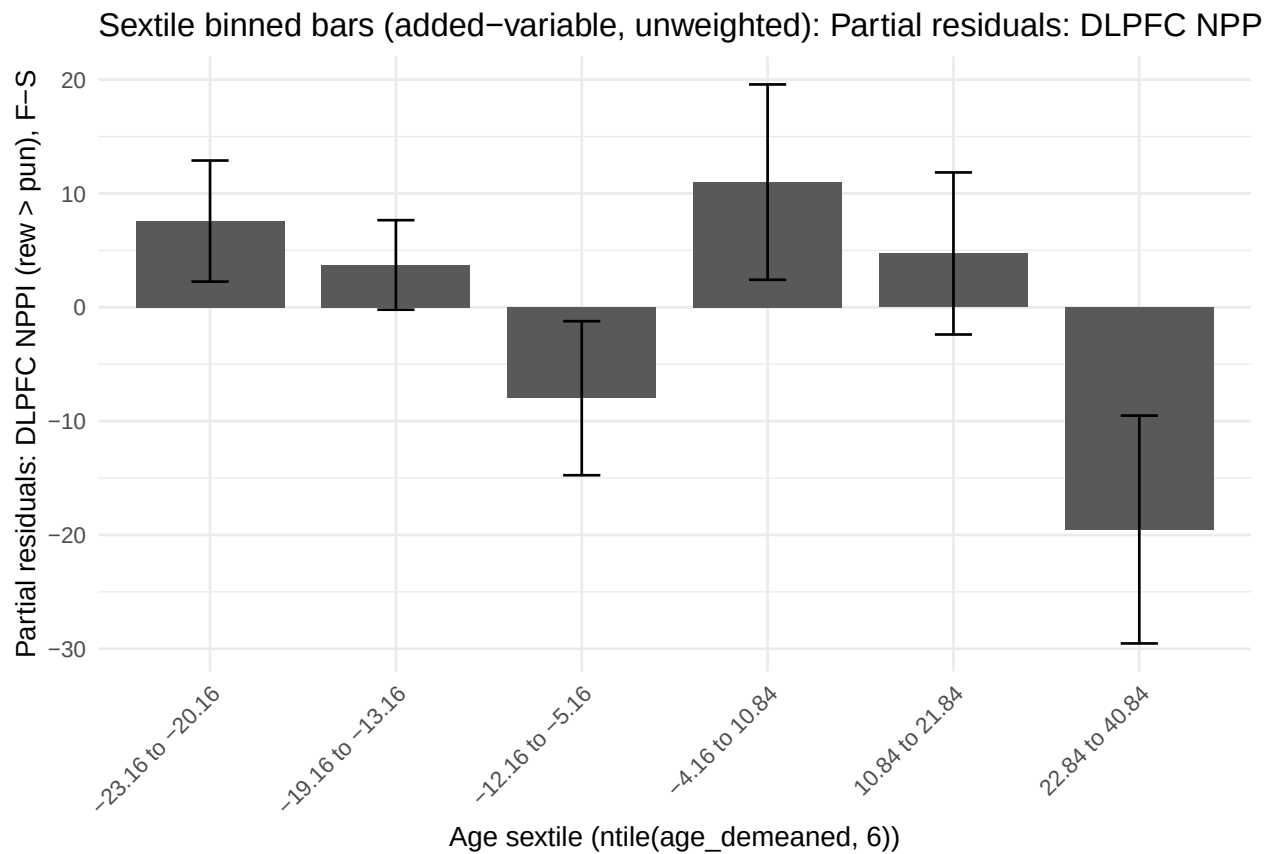
```
sum_raw_w <- summarize_sextiles(df, y = df[[outcome_dlpfc]], series_label = "Weighted mean", w = df[[w_
plot_sextile_bars(sum_raw_w, outcome_label = label_dlpfc, title_prefix = "Sextile binned bars (raw, wei
```



Bars show mean within sextile; error bars are ± 1 SEM. Selection-biased (circular).

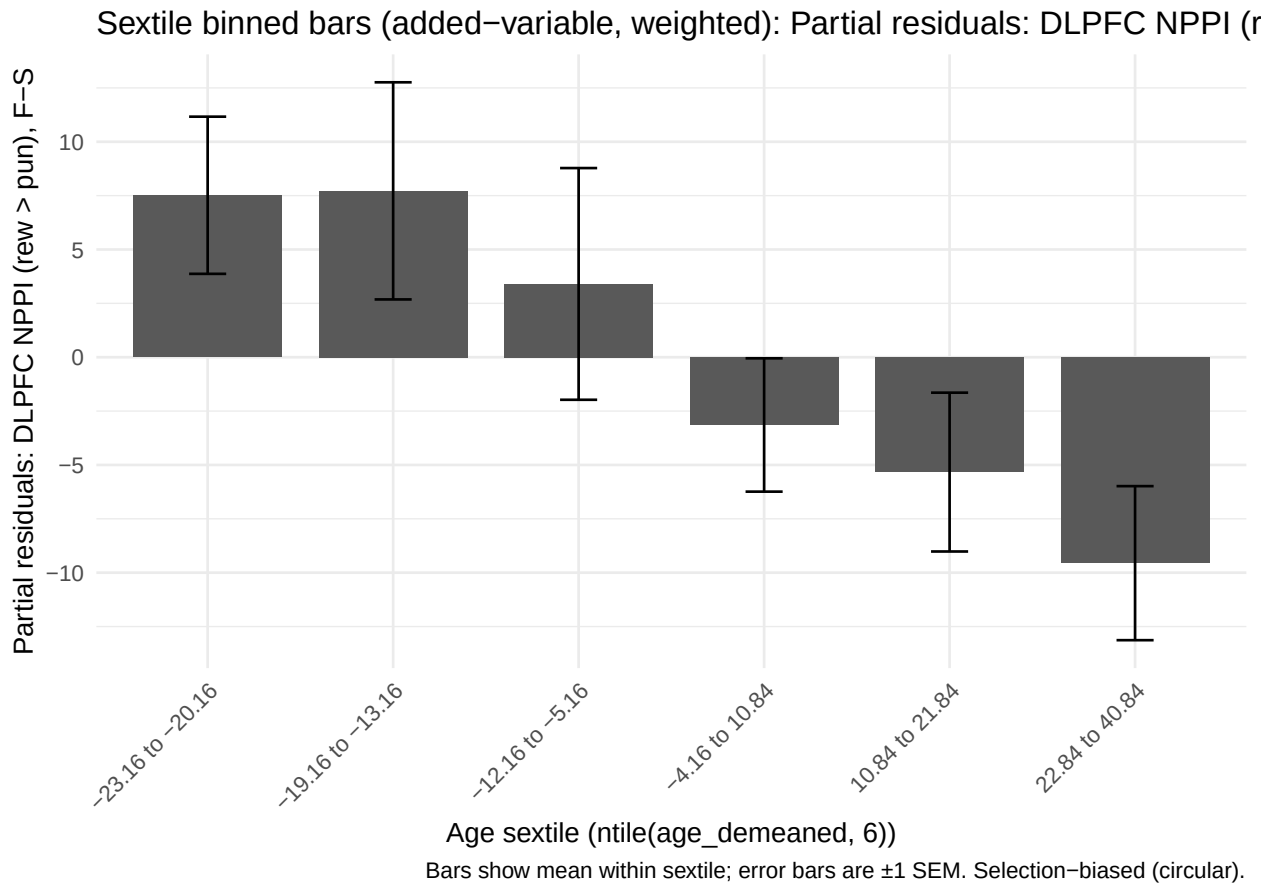
```
# 3b) Added-variable outcome (partial residuals differ for weighted vs unweighted models)
pr_unw <- partial_residual_age(model2_unw, df)
pr_w   <- partial_residual_age(model2_w, df)

sum_pr_unw <- summarize_sextiles(df, y = pr_unw, series_label = "Unweighted model PR")
plot_sextile_bars(sum_pr_unw, outcome_label = paste0("Partial residuals: ", label_dlpfc),
  title_prefix = "Sextile binned bars (added-variable, unweighted)")
```



Bars show mean within sextile; error bars are ± 1 SEM. Selection-biased (circular).

```
sum_pr_w <- summarize_sextiles(df, y = pr_w, series_label = "Weighted model PR", w = df[[w_dlpfc]])
plot_sextile_bars(sum_pr_w, outcome_label = paste0("Partial residuals: ", label_dlpfc),
  title_prefix = "Sextile binned bars (added-variable, weighted)")
```



4. MTG ACT (rew), Friend-Stranger

Outcome: mask.mtg_type.act_cnum.17_cname.rew_F.S.txt

Variance: var_mask.mtg_type.act_cnum.17_cname.rew_F.S.txt

Main predictor: age_demeaned

Covariates: sex + avg_tsnr_demeaned + avg_fdmean_demeaned + flip_demeaned + study_demeaned

---- MTG model (robust to name-repair on import) ----

```
outcome_mtg <- find_one_col("^mask[.-]mtg_type[.-]act_cnum[.-]17_cname[.-]rew_F[.-]S[.-]txt([.]{3}[0-9])")
```

```
var_mtg <- find_one_col("^var_mask[.-]mtg_type[.-]act_cnum[.-]17_cname[.-]rew_F[.-]S[.-]txt([.]{3}[0-9])")
```

```
w_mtg <- "weights_mtg_type.act_cnum.17_cname.rew_F.S.txt"
```

```
needed_mtg <- c(outcome_mtg, var_mtg,
               "age_demeaned", "sex", "avg_tsnr_demeaned", "avg_fdmean_demeaned",
               "flip_demeaned", "study_demeaned")
```

```
missing_mtg <- setdiff(needed_mtg, names(df))
```

```
if (length(missing_mtg) > 0) stop(paste("Missing required columns:", paste(missing_mtg, collapse = ", ")))
```

```
df[[w_mtg]] <- make_invvar_weights(df[[var_mtg]], var_floor = 1e-6)
```

```
print(summarize_weights(df[[w_mtg]]))
```

```
## # A tibble: 1 x 9
```

	n	n_nonfinite	min	p01	p05	median	p95	p99	max
<int>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	245	0	0.00000517	9.01e-6	1.69e-5	5.28e-5	0.00382	0.00615	0.00934

```
fml_mtg <- as.formula(sprintf(
  "%s` ~ age_demeaned + sex + avg_tsnr_demeaned + avg_fdmean_demeaned + flip_demeaned + study_demeaned
  outcome_mtg
  ))

model3_w <- lm(fml_mtg, data = df, weights = df[[w_mtg]])
model3_unw <- lm(fml_mtg, data = df)

summary(model3_w)
```

```
##
## Call:
## lm(formula = fml_mtg, data = df, weights = df[[w_mtg]])
##
## Weighted Residuals:
##      Min       1Q   Median       3Q      Max
## -2.97980 -0.57492 -0.05937  0.58722  2.69874
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -18.6444     8.1612  -2.285  0.0232 *
## age_demeaned     0.7758     0.1833   4.232 3.32e-05 ***
## sexM             2.3169     6.2183   0.373  0.7098
## sex0            -3.5306    55.5441  -0.064  0.9494
## avg_tsnr_demeaned -0.1952     0.2266  -0.861  0.3899
## avg_fdmean_demeaned -128.6526    63.8628  -2.015  0.0451 *
## flip_demeaned    10.7033    33.6496   0.318  0.7507
## study_demeaned   -41.4942    34.2712  -1.211  0.2272
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9309 on 237 degrees of freedom
## Multiple R-squared:  0.08899,    Adjusted R-squared:  0.06209
## F-statistic: 3.307 on 7 and 237 DF,  p-value: 0.002237
```

```
summary(model3_unw)
```

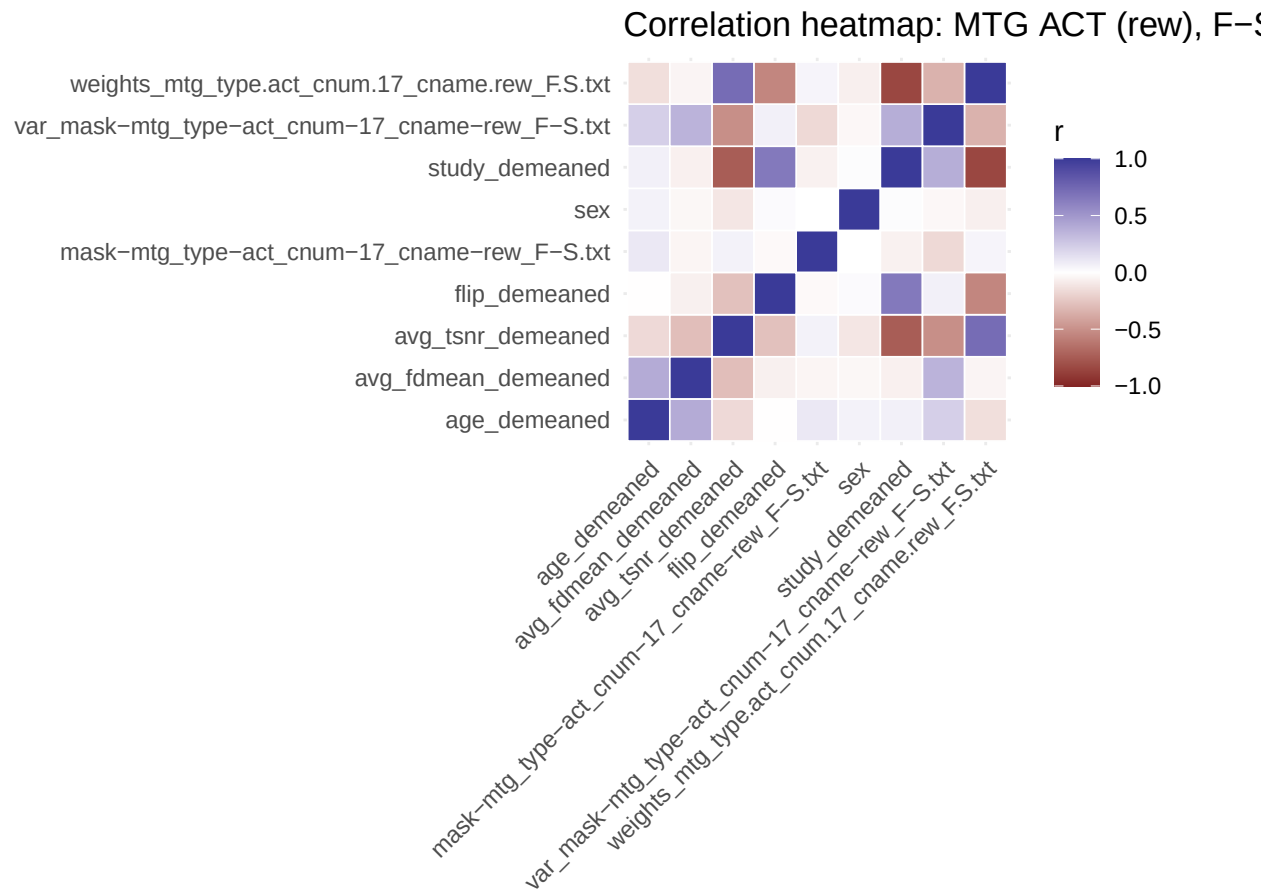
```
##
## Call:
## lm(formula = fml_mtg, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -409.65  -75.83   -1.05    73.73   587.96
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -21.6883    11.9494  -1.815  0.0708 .
## age_demeaned     1.2159     0.5688   2.138  0.0336 *
## sexM            -0.5161    19.7508  -0.026  0.9792
## sex0            -38.4505    73.2140  -0.525  0.5999
## avg_tsnr_demeaned -0.3095     1.2113  -0.256  0.7985
## avg_fdmean_demeaned -206.4011    143.3476  -1.440  0.1512
## flip_demeaned     15.7150    31.7619   0.495  0.6212
```

```
## study_demeaned      -56.5261    66.5846  -0.849   0.3968
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 143.2 on 237 degrees of freedom
## Multiple R-squared:  0.02754,    Adjusted R-squared:  -0.00118
## F-statistic: 0.9589 on 7 and 237 DF,  p-value: 0.462
```

4.1 Plots for MTG

```
label_mtg <- "MTG ACT (rew), F-S"

# 0) Correlation heatmap among outcome + predictors (+ variance + weight)
# 0a) Correlation heatmap (all correlations; no cell labels)
plot_corr_heatmap(
  df,
  cols = c(outcome_mtg, "age_demeaned", "sex", "avg_tsnr_demeaned", "avg_fdmean_demeaned",
            "flip_demeaned", "study_demeaned", var_mtg, w_mtg),
  title = paste0("Correlation heatmap: ", label_mtg),
  threshold = FALSE,
  show_numbers = "none"
)
```

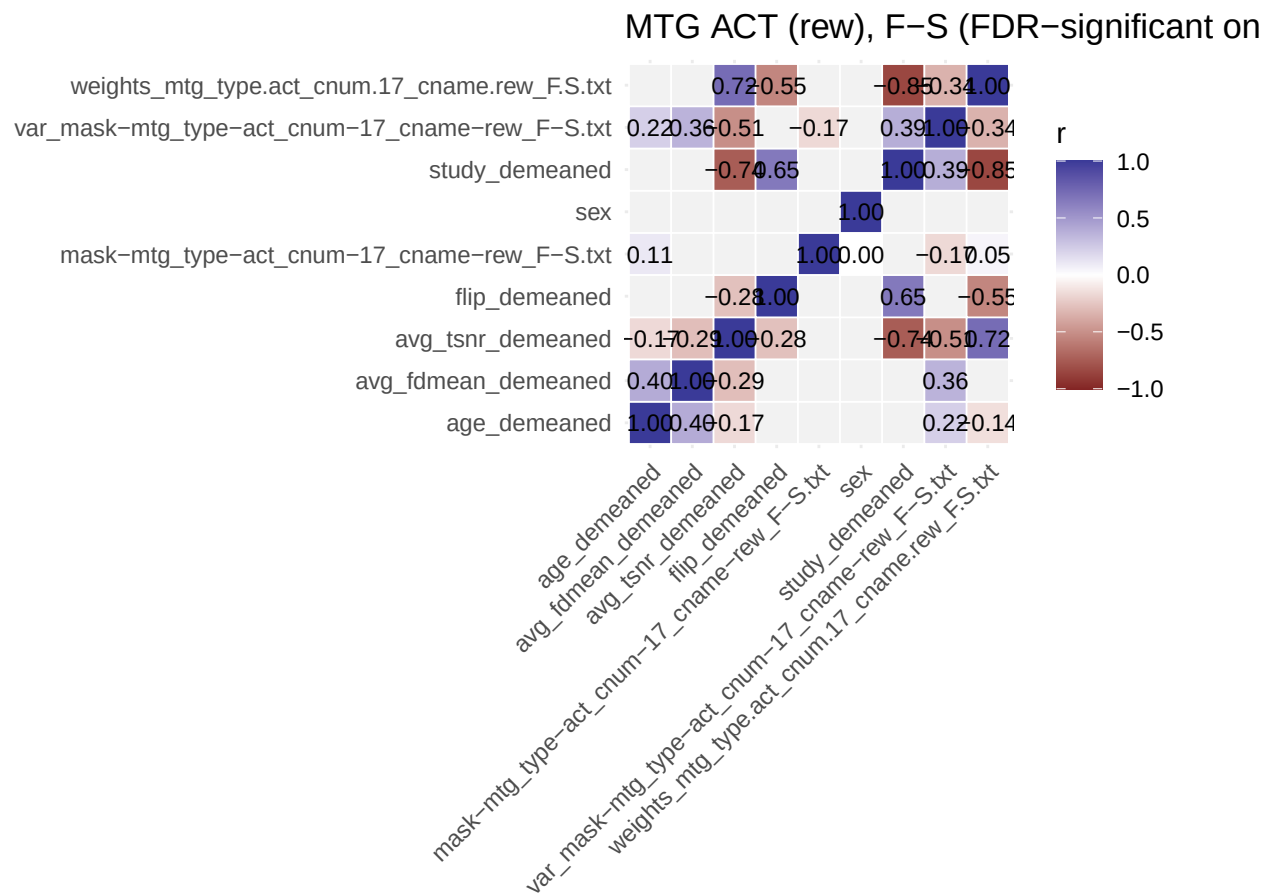


All pairwise correlations shown (no multiple-comparisons thresholding).

```

# 0b) Correlation heatmap (FDR-significant only; labels shown for significant cells)
plot_corr_heatmap(
  df,
  cols = c(outcome_mtg, "age_demeaned", "sex", "avg_tsnr_demeaned", "avg_fdmean_demeaned",
            "flip_demeaned", "study_demeaned", var_mtg, w_mtg),
  title = paste0(label_mtg, " (FDR-significant only)",
  threshold = TRUE,
  show_numbers = "significant"
)

```

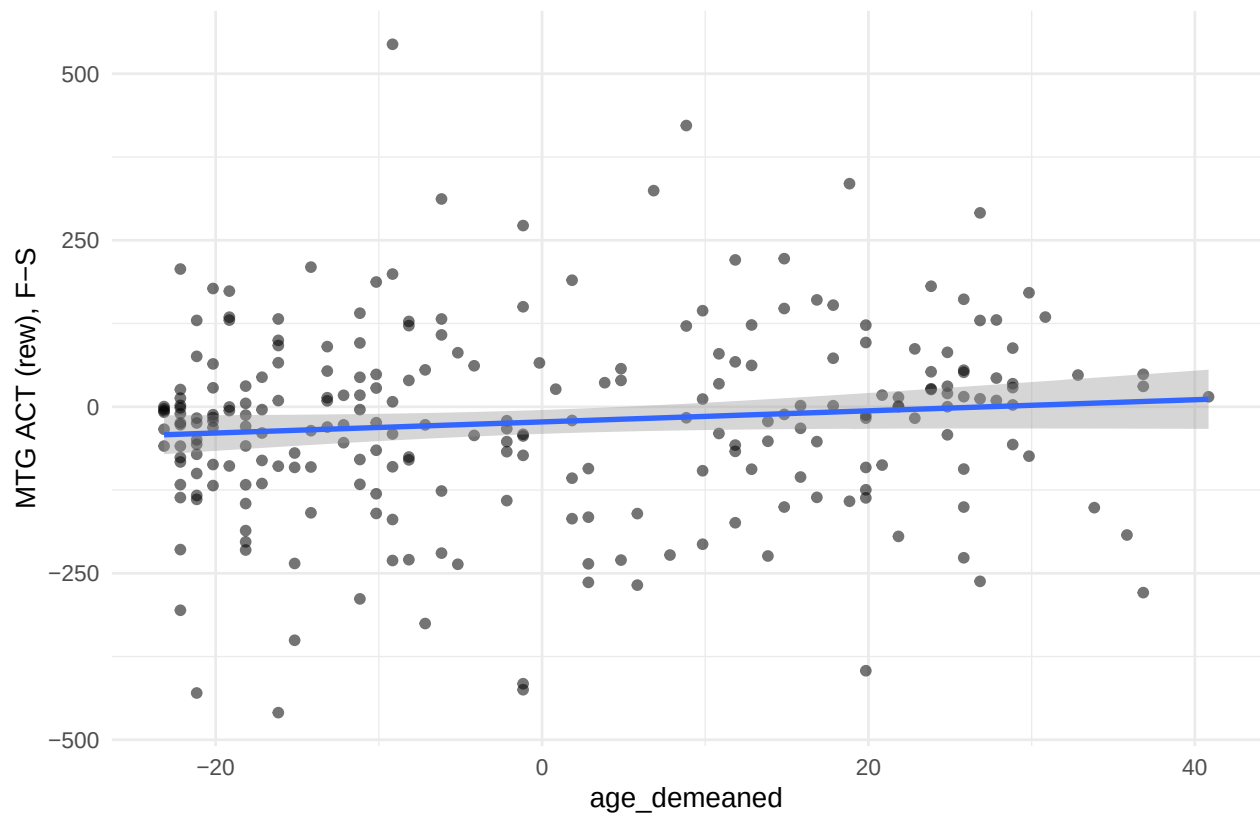


```

# 1) Raw scatter: unweighted vs weighted
plot_scatter_raw_unweighted(df, outcome_mtg, label_mtg)

```

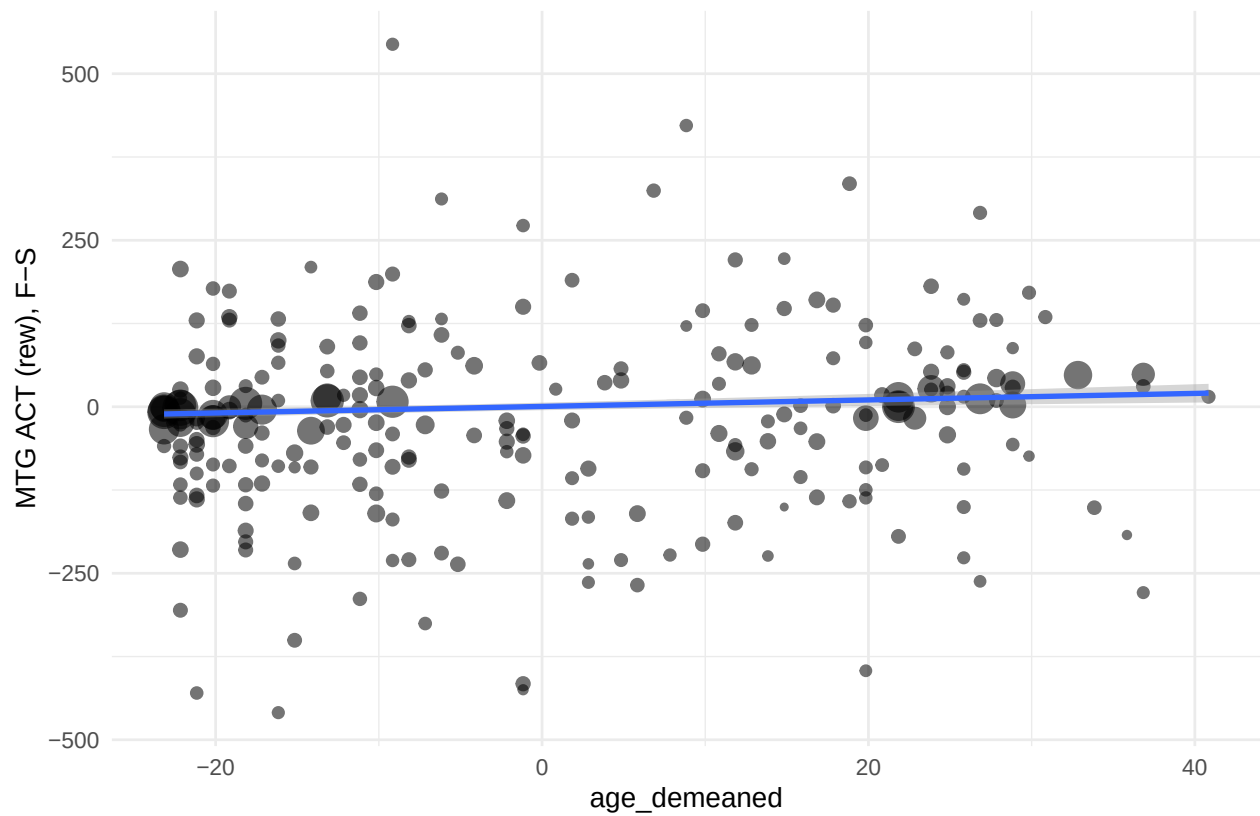
Raw scatter (unweighted): MTG ACT (rew), F-S



Selection-biased (circular) visualization; do not treat as an independent effect size.

```
plot_scatter_raw_weighted(df, outcome_mtg, w_mtg, label_mtg)
```

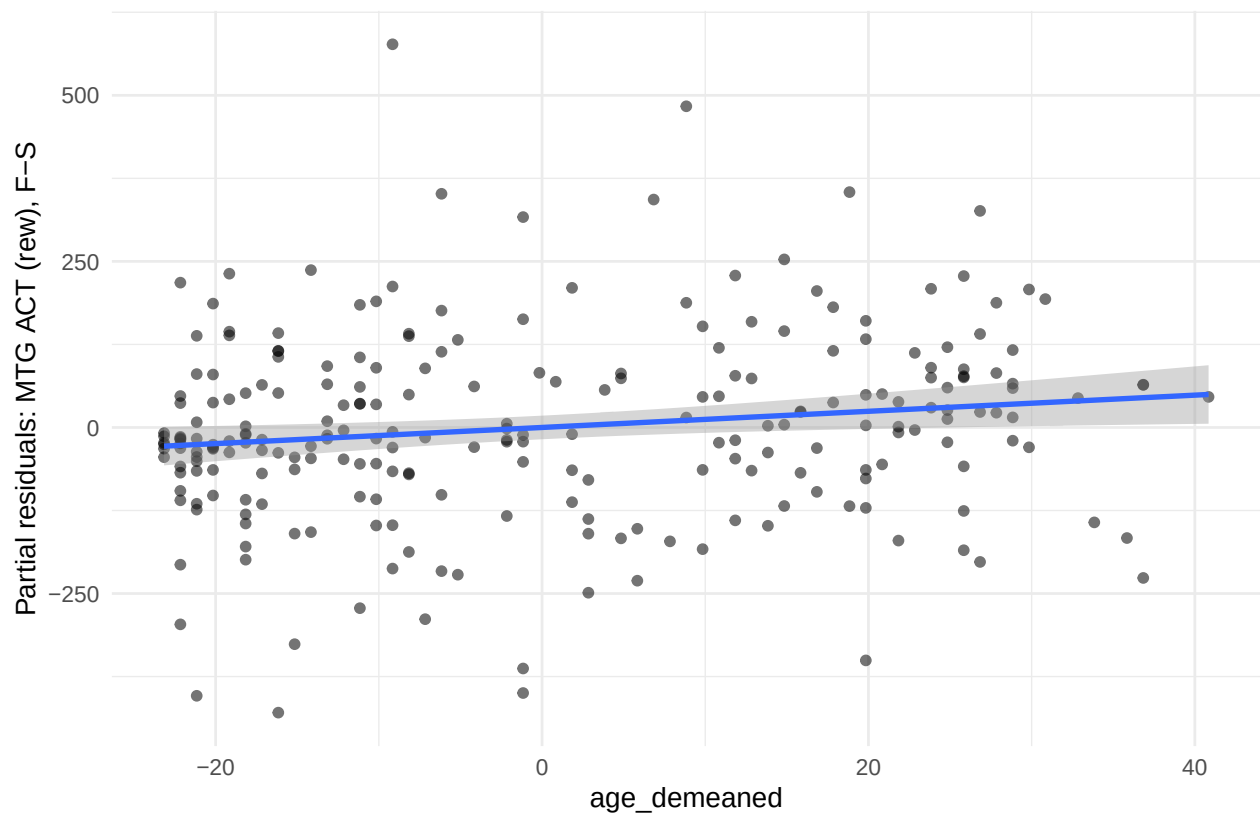
Raw scatter (weighted): MTG ACT (rew), F-S



Point size reflects inverse-variance weight; smooth uses the same weights. Selection-biased (circular).

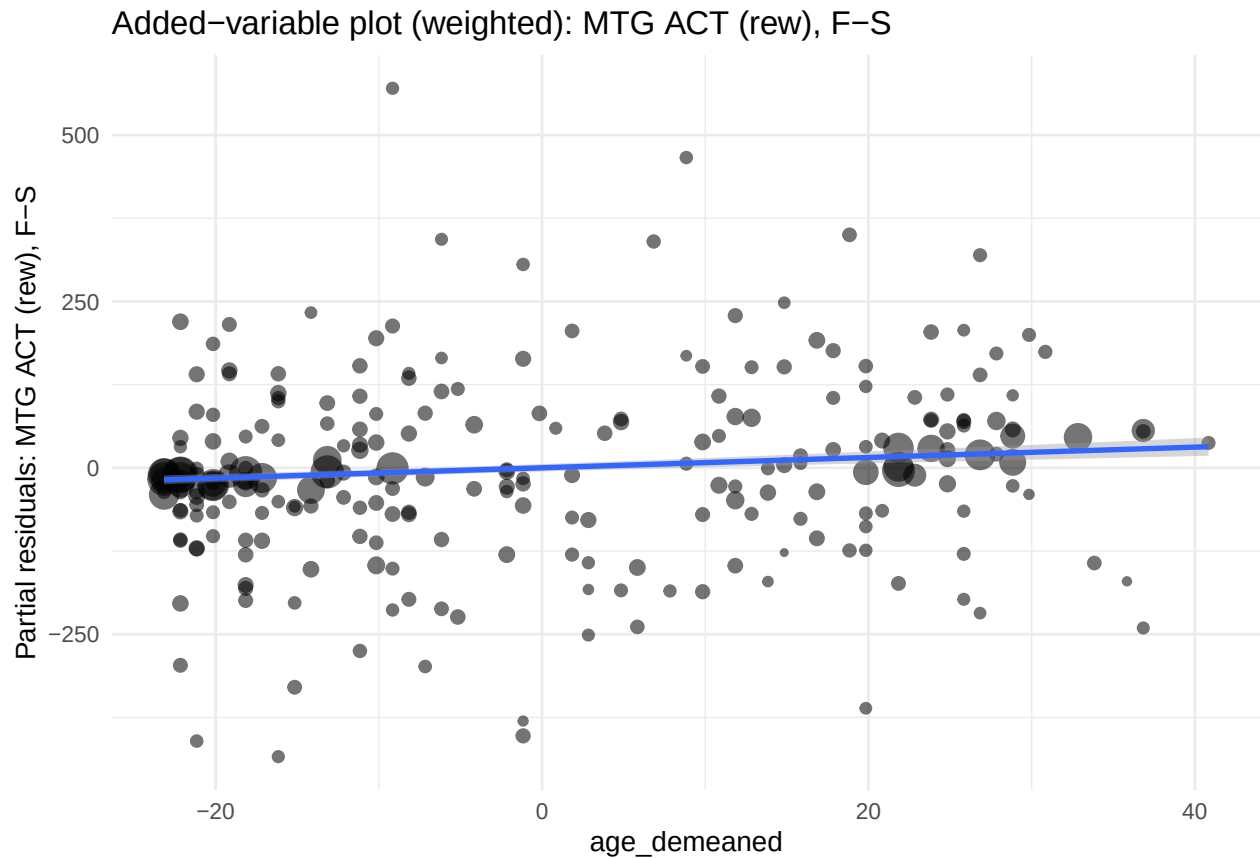
```
# 2) Added-variable (partial residual) plots: unweighted vs weighted  
plot_added_variable_unweighted(model3_unw, df, label_mtg)
```

Added-variable plot (unweighted): MTG ACT (rew), F-S



Partial residuals are computed from the unweighted model. Selection-biased (circular).

```
plot_added_variable_weighted(model3_w, df, w_mtg, label_mtg)
```

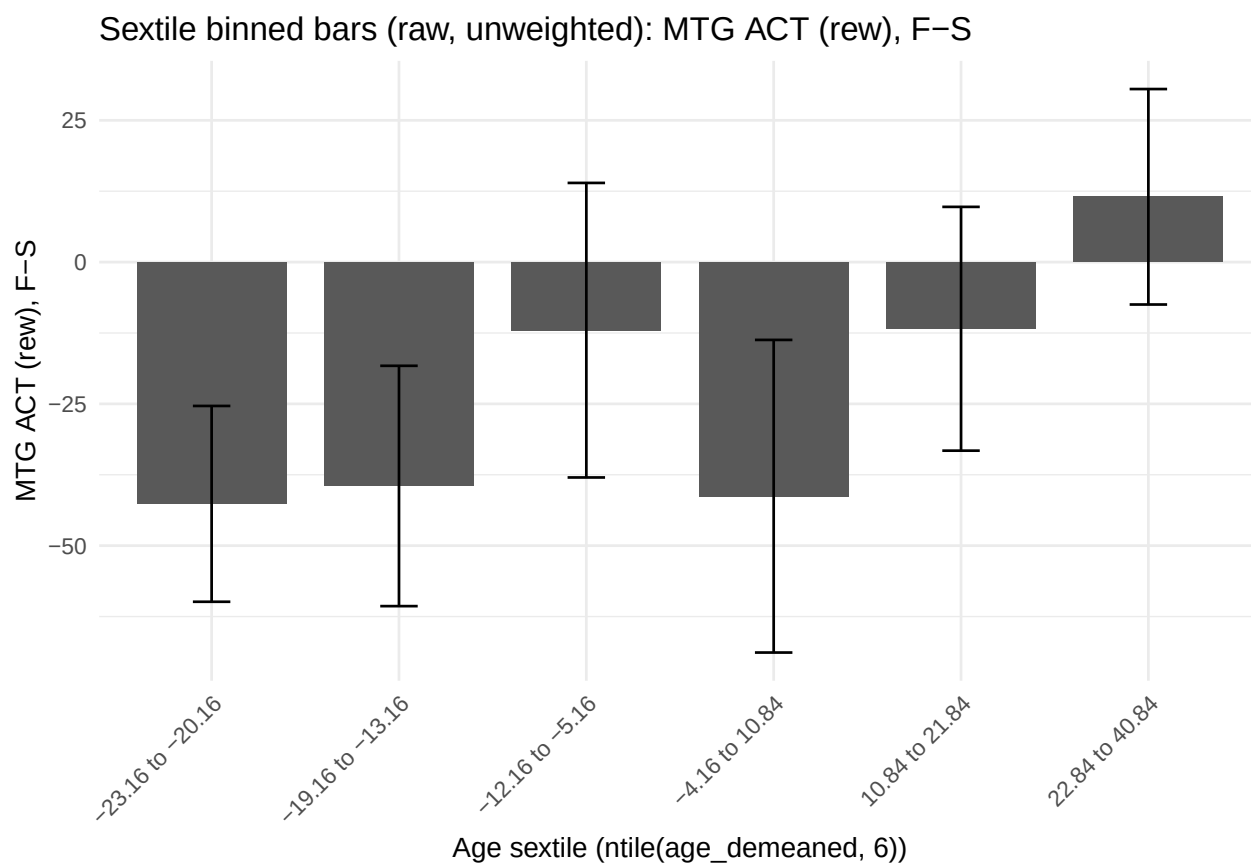


3) Bar plots with age binned into sextiles (6 equal bins)

3a) Raw outcome bars (unweighted and weighted shown separately)

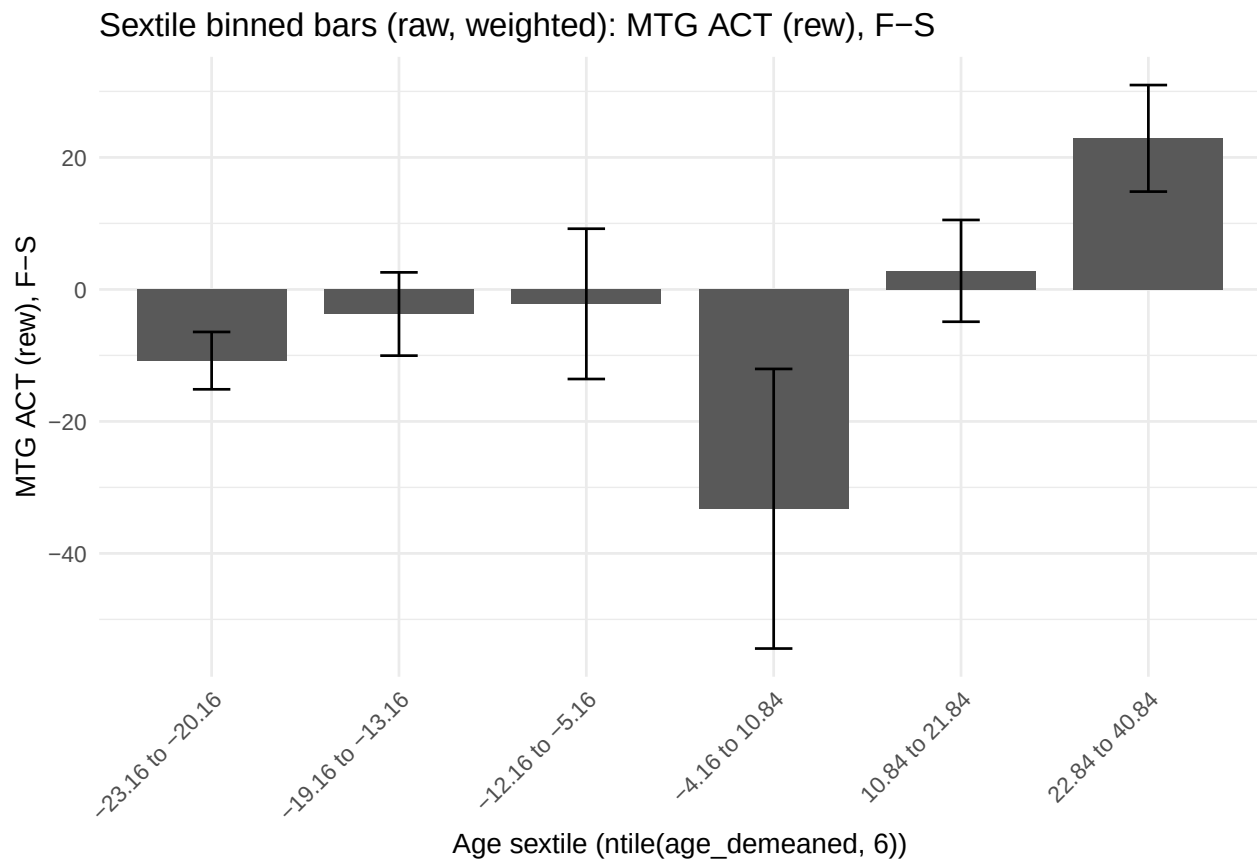
```
sum_raw_unw <- summarize_sextiles(df, y = df[[outcome_mtg]], series_label = "Unweighted mean")
```

```
plot_sextile_bars(sum_raw_unw, outcome_label = label_mtg, title_prefix = "Sextile binned bars (raw, unw")
```



Bars show mean within sextile; error bars are ± 1 SEM. Selection-biased (circular).

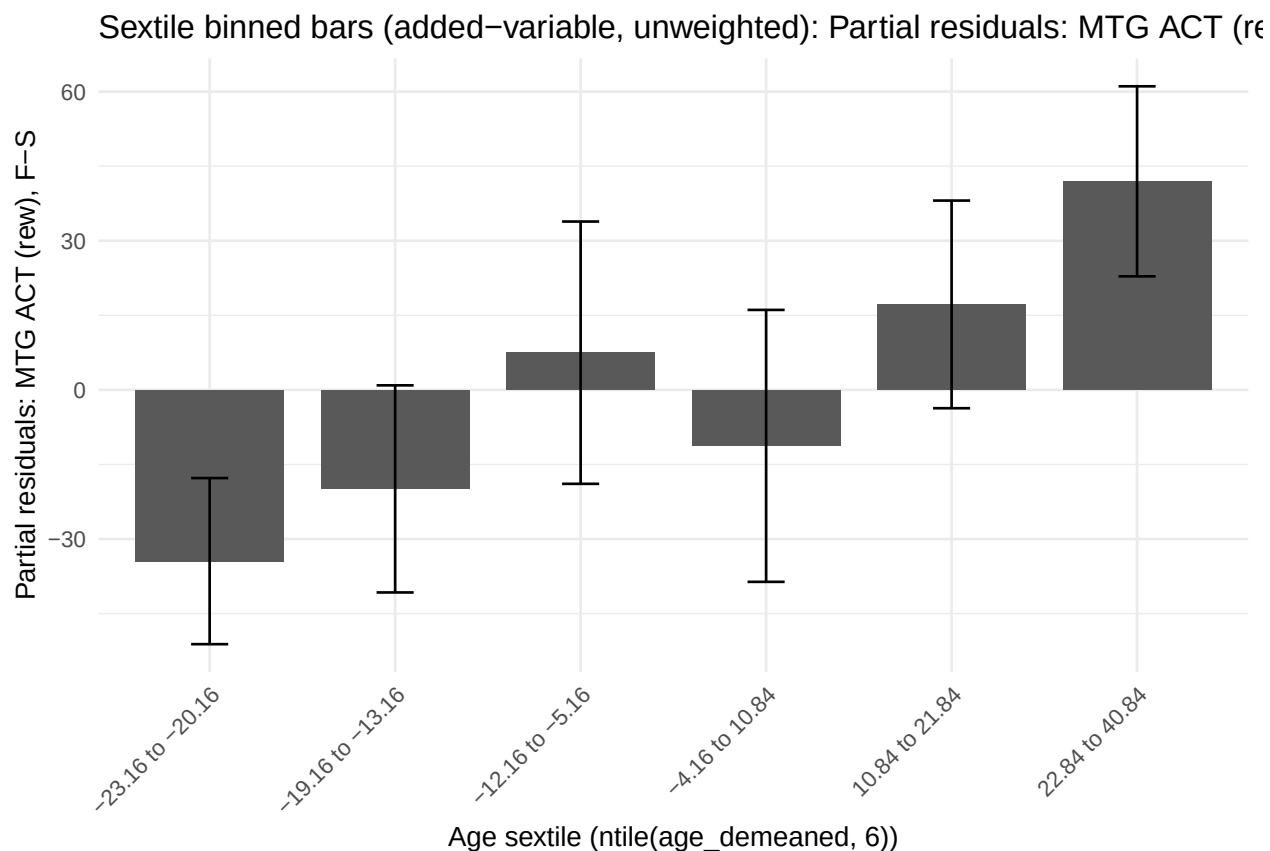
```
sum_raw_w <- summarize_sextiles(df, y = df[[outcome_mtg]], series_label = "Weighted mean", w = df[[w_mtg]],
plot_sextile_bars(sum_raw_w, outcome_label = label_mtg, title_prefix = "Sextile binned bars (raw, weigh
```



Bars show mean within sextile; error bars are ± 1 SEM. Selection-biased (circular).

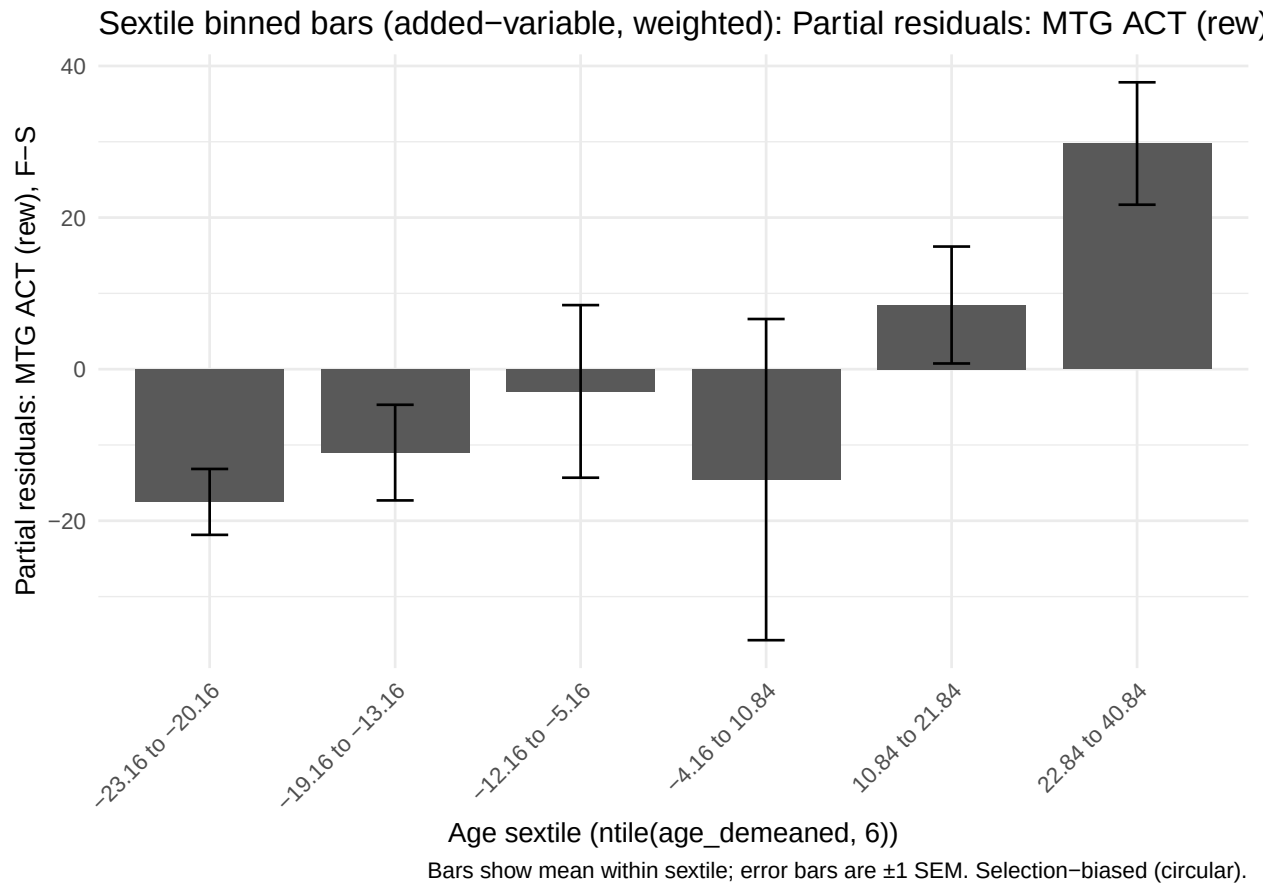
```
# 3b) Added-variable outcome (partial residuals differ for weighted vs unweighted models)
pr_unw <- partial_residual_age(model3_unw, df)
pr_w   <- partial_residual_age(model3_w, df)

sum_pr_unw <- summarize_sextiles(df, y = pr_unw, series_label = "Unweighted model PR")
plot_sextile_bars(sum_pr_unw, outcome_label = paste0("Partial residuals: ", label_mtg),
  title_prefix = "Sextile binned bars (added-variable, unweighted)")
```



Bars show mean within sextile; error bars are ± 1 SEM. Selection-biased (circular).

```
sum_pr_w <- summarize_sextiles(df, y = pr_w, series_label = "Weighted model PR", w = df[[w_mtg]])
plot_sextile_bars(sum_pr_w, outcome_label = paste0("Partial residuals: ", label_mtg),
  title_prefix = "Sextile binned bars (added-variable, weighted)")
```



5. Session info

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.2
##
## Matrix products: default
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/Libraries/libBLAS.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats graphics grDevices utils datasets methods base
##
## other attached packages:
## [1] tibble_3.3.0 readr_2.1.6 ggplot2_4.0.1 dplyr_1.1.4
##
## loaded via a namespace (and not attached):
```

## [1] Matrix_1.7-4	bit_4.6.0	gtable_0.3.6	compiler_4.5.2
## [5] crayon_1.5.3	tinytex_0.58	tidyselect_1.2.1	parallel_4.5.2
## [9] splines_4.5.2	scales_1.4.0	yaml_2.3.12	fastmap_1.2.0
## [13] lattice_0.22-7	R6_2.6.1	labeling_0.4.3	generics_0.1.4
## [17] knitr_1.50	pillar_1.11.1	RColorBrewer_1.1-3	tzdb_0.5.0
## [21] rlang_1.1.6	xfun_0.55	S7_0.2.1	bit64_4.6.0-1
## [25] cli_3.6.5	withr_3.0.2	magrittr_2.0.4	mgcv_1.9-3
## [29] digest_0.6.39	grid_4.5.2	vroom_1.6.7	rstudioapi_0.17.1
## [33] hms_1.1.4	nlme_3.1-168	lifecycle_1.0.4	vctrs_0.6.5
## [37] evaluate_1.0.5	glue_1.8.0	farver_2.1.2	rmarkdown_2.30
## [41] tools_4.5.2	pkgconfig_2.0.3	htmltools_0.5.9	